CAPACITACIÓN QA AUTOMATION · GUÍA COMPLEMENTARIA

MANUAL DE GIT Y GITHUB

Git y GitHub desde cero para entregar tus ejercicios

De una PC sin Git a tu primer Pull Request revisado: instalar, configurar tu identidad, conectar tu cuenta de GitHub y trabajar con ramas como en un equipo real.

Git
historial de cambios en tu PC

GitHub
el repositorio compartido

gh CLI
inicio de sesión sin contraseñas

Pull Request
revisión y entrega

01 Antes de empezar: las cinco palabras clave
02 Paso 1 · Crear tu cuenta de GitHub
03 Paso 2 · Instalar Git
04 Paso 3 · Configurar tu identidad
05 Paso 4 · Conectar tu PC con GitHub
06 Paso 5 · Clonar el repositorio del curso
07 Paso 6 · El flujo de cada ejercicio
08 Paso 7 · Pull Request y revisión
09 Resolver un conflicto
10 Errores típicos y su solución
11 Chuleta de comandos y checklist

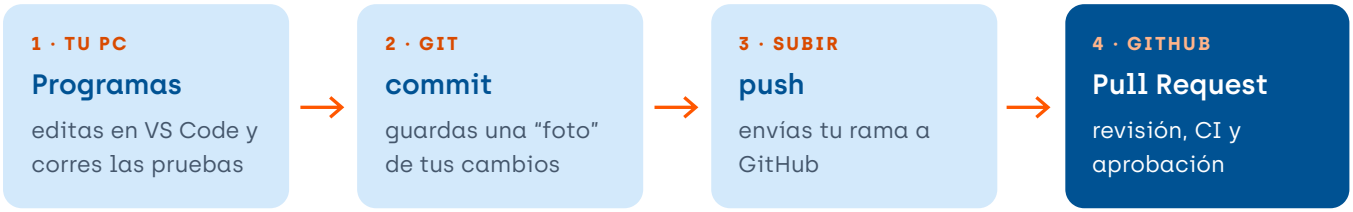
Instructor: **Giancarlo Palomino** · QA Automation Engineer

Windows · macOS · Linux · v1.0 · octubre 2026

MAPA DEL TRABAJO

Qué es Git, qué es GitHub y cómo se conectan

Piensa en un documento compartido con control de versiones: **Git** es el cuaderno donde anotas cada cambio que haces en tu PC; **GitHub** es la oficina donde se guarda la copia oficial y donde el instructor revisa tu trabajo antes de aceptarlo.



PALABRA	QUÉ SIGNIFICA	EJEMPLO EN EL CURSO
Repositorio	La carpeta del proyecto con todo su historial de cambios.	`canvia-restassured-bdd`
Commit	Una "foto" de tus cambios con un mensaje que explica qué hiciste.	`feat: agrega escenario de login`
Rama (branch)	Una línea de trabajo paralela. Trabajas en la tuya sin tocar la de los demás.	`feature/ejercicio-1-ana-torres`
Push / Pull	*Push* sube tus commits a GitHub; *pull* trae los commits nuevos a tu PC.	`git push` · `git pull`
Pull Request	El pedido para que tus cambios entren a la rama principal. Ahí se revisan.	PR de tu rama hacia `develop`

Las dos ramas del curso

DEVELOP · RAMA DE TRABAJO

Aquí llegan todos los ejercicios aprobados. Siempre creas tu rama **desde** `develop` y tu Pull Request va **hacia** `develop`.

MAIN · VERSIÓN ESTABLE

Solo el instructor la actualiza al cerrar un bloque del curso. Nunca trabajes ni hagas push directo en `main`.

NINGUNA DE LAS DOS ACEPTA PUSH DIRECTO

Ambas ramas están protegidas: todo cambio entra por Pull Request, con la CI en verde y la aprobación del instructor. Si intentas un `git push` a `develop`, GitHub lo rechaza. No es un error tuyo: es la regla.

IDENTIDAD EN LA WEB

1 Crear tu cuenta de GitHub

1. Entra a `github.com/signup` y crea la cuenta con tu correo.
2. Elige un usuario profesional, por ejemplo `ana-torres`: lo verán tus compañeros y futuros empleadores.
3. Confirma el correo con el código que te llega.
4. Activa la verificación en dos pasos en *Settings* → *Password and authentication*.
5. Envía tu usuario al instructor para que te dé acceso al repositorio.

¿YA TIENES CUENTA PERSONAL?

Puedes usarla. Solo verifica que el correo con el que harás commits esté agregado en *Settings* → *Emails*; así tus commits aparecen enlazados a tu perfil.

HERRAMIENTA EN TU PC

2 Instalar Git

Windows 10 / 11

1 · INSTALAR

```
# PowerShell
winget install --id Git.Git -e
```

O el instalador de `git-scm.com/download/win`: deja **todas** las opciones por defecto (incluye Git Credential Manager).

2 · VERIFICAR (TERMINAL NUEVA)

```
git --version
```

macOS INTEL / APPLE SILICON

1 · INSTALAR

```
# herramientas de Apple (trae Git)
xcode-select --install
# o con Homebrew
brew install git
```

2 · VERIFICAR

```
git --version
```

Linux UBUNTU · DEBIAN · FEDORA

1 · INSTALAR

```
# Ubuntu / Debian
sudo apt update
sudo apt install git
# Fedora / RHEL
sudo dnf install git
```

2 · VERIFICAR

```
git --version
```

```
# Resultado esperado
$ git --version
git version 2.5x.x
```

SI DICE "NO SE RECONOCE"

La terminal se abrió antes de instalar. Ciérrala y abre una nueva. En Windows, si sigue fallando, reinicia la sesión de usuario.

UNA SOLA VEZ POR PC

3 Configurar tu identidad en Git

Git firma cada commit con un nombre y un correo. Usa **el mismo correo de tu cuenta de GitHub**: así cada commit aparece con tu foto y enlazado a tu perfil. Estos comandos son iguales en los tres sistemas.

Terminal — OBLIGATORIO

```
git config --global user.name "Ana Torres"
git config --global user.email "ana.torres@ejemplo.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
git config --global core.editor "code --wait"
```

CONFIGURACIÓN	PARA QUÉ SIRVE
`user.name` / `user.email`	Quién hizo cada commit. Sin esto Git se niega a guardar: *Author identity unknown*.
`init.defaultBranch main`	Los repositorios nuevos empiezan en `main`, como en GitHub.
`pull.rebase false`	`git pull` une los cambios con un *merge*, el comportamiento más fácil de entender.
`core.editor "code --wait"`	Cuando Git necesita un texto largo (mensaje, conflicto), abre VS Code en lugar de un editor de consola.

Saltos de línea: depende del sistema

Windows — POWERSHELL

```
git config --global core.autocrlf true
```

macOS / Linux — TERMINAL

```
git config --global core.autocrlf input
```

Windows y macOS/Linux terminan las líneas de forma distinta. Sin este ajuste, un archivo que no tocaste aparece como "modificado entero" y ensucia tu Pull Request.

```
# Revisa lo que configuraste
$ git config --global --list
user.name=Ana Torres
user.email=ana.torres@ejemplo.com
init.defaultbranch=main
pull.rebase=false
core.editor=code --wait
```

¿TE EQUIVOCASTE?

Vuelve a ejecutar el mismo comando con el valor correcto: el nuevo reemplaza al anterior. No hace falta borrar nada.

AUTENTICACIÓN

4 Conectar tu PC con tu cuenta de GitHub

Desde 2021 GitHub **no acepta tu contraseña** en la terminal. Elige una de estas tres opciones; la A es la más simple y la que usaremos en clase.

OPCIÓN A · RECOMENDADA · GITHUB CLI

Abre el navegador, apruebas y queda listo para Git y para crear Pull Requests desde la terminal.

Windows — POWERSHELL

```
winget install --id GitHub.cli -e
```

macOS — HOMEBREW

```
brew install gh
```

Linux — UBUNTU · DEBIAN

```
sudo apt install gh
```

```
# En una terminal nueva
$ gh auth login
? Where do you use GitHub?          GitHub.com
? Preferred protocol?               HTTPS
? Authenticate Git with credentials? Yes
? How would you like to authenticate? Login with a web browser
! First copy your one-time code: AB12-CD34
# pega el código en el navegador y autoriza
$ gh auth status
✓ Logged in to github.com account ana-torres
```

QUÉ ACABA DE PASAR

`gh` guardó un permiso seguro en tu PC y le dijo a Git que lo use. Desde ahora `git clone`, `git push` y `gh pr create` funcionan sin pedirte nada.

OPCIÓN B · SOLO WINDOWS · CREDENTIAL MANAGER

Viene con Git para Windows. La primera vez que hagas `git push` se abre una ventana para iniciar sesión en GitHub. Después no vuelve a preguntar.

OPCIÓN C · LLAVE SSH

Para redes que bloquean el inicio por navegador. Crea la llave, súbela a *Settings → SSH and GPG keys* y clona con la URL `git@github.com:…`

Opción C · Llave SSH — TERMINAL

```
ssh-keygen -t ed25519 -C "ana.torres@ejemplo.com"     # Enter a todo
Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | Set-Clipboard   # Windows: copia la llave pública
pbcopy < ~/.ssh/id_ed25519.pub                        # macOS
ssh -T git@github.com                                 # prueba: "Hi ana-torres! You've successfully authenticated"
```

NUNCA COMPARTAS

La llave privada (`id_ed25519` sin `.pub`), tokens ni contraseñas: ni en el chat, ni en el código, ni en una captura.

UNA SOLA VEZ

5 Clonar el repositorio del curso

Terminal — CLONAR

```
mkdir C:\cursos
cd C:\cursos
git clone https://github.com/gpalominoh-canvia/canvia-restassured-bdd.git
cd canvia-restassured-bdd
code .
```

CARPETA SIN ESPACIOS NI TILDES

Rutas como `C:\Users\José\Mis Documentos` causan errores raros en Maven y Node. Usa algo corto como `C:\cursos`.

CADA EJERCICIO

6 El flujo de trabajo: rama, commit, push

A develop al día — `git pull`

→

B tu rama — `git switch -c`

→

C programar — y correr las pruebas

→

D commit — `git add` + `commit`

→

E push — tu rama a GitHub

Terminal — UN EJERCICIO COMPLETO

```
# a · traer lo último de develop
git switch develop
git pull

# b · crear tu rama: feature/<ejercicio>-<tu-nombre>, en minúsculas y con guiones
git switch -c feature/ejercicio-1-ana-torres

# c · programar y probar... luego revisar qué cambió
git status
git diff

# d · elegir qué guardar y hacer el commit
git add src/
git commit -m "feat: agrega escenario de reserva con precio negativo"

# e · subir la rama (la primera vez con -u; después basta git push)
git push -u origin feature/ejercicio-1-ana-torres
```

REGLA DE ORO

Antes de cada commit, corre las pruebas. Si no pasan en tu PC, tampoco pasarán en la CI de GitHub.

MENSAJES DE COMMIT

Conventional Commits, en español

PREFIJO	CUÁNDO USARLO	EJEMPLO
`feat:`	Agregas un escenario, step o funcionalidad.	`feat: valida reserva sin fecha de salida`
`fix:`	Corriges algo que fallaba.	`fix: corrige selector del botón login`
`refactor:`	Reordenas código sin cambiar lo que hace.	`refactor: extrae datos de prueba`
`test:`	Solo cambias pruebas o datos de prueba.	`test: agrega ejemplos al esquema`
`docs:`	Documentación o README.	`docs: explica cómo correr @smoke`

Haz commits pequeños, uno por idea. Cinco commits claros se revisan mejor que uno gigante.

ENTREGA

7 Abrir el Pull Request y atender la revisión

DESDE LA WEB

1. Tras el `push`, GitHub muestra el botón **Compare & pull request**.
2. Verifica: **base: develop** ← **compare: tu rama**.
3. Título con el formato de commit y, en la descripción, `Closes #N` (número del Issue del ejercicio).
4. Clic en **Create pull request** y espera la CI.

DESDE LA TERMINAL

Terminal — GH

```
gh pr create --base develop \
  --title "feat: ejercicio 1 - Ana Torres" \
  --body "Closes #1"
gh pr checks      # estado de la CI
```

CI EN VERDE
GitHub corrió las pruebas en un servidor limpio y pasaron. Tu PR queda listo para revisión.

CI EN ROJO
Abre el detalle (*Details*), lee el error, corrígelo en tu rama y haz `push`. El PR se actualiza solo.

CAMBIOS PEDIDOS
No abras otro PR. Sigue en la misma rama: corrige, `commit`, `push`.

Terminal — TRAER CAMBIOS NUEVOS DE DEVELOP A TU RAMA

```
git switch develop
git pull
git switch feature/ejercicio-1-ana-torres
git merge develop        # si aparece un conflicto, ver la página siguiente
```

SIN PÁNICO

Resolver un conflicto

Un conflicto ocurre cuando tú y otra persona cambiaron **las mismas líneas**. Git no adivina cuál versión vale y te pide decidir. No se pierde nada.

reservas.feature — ASÍ LO MARCA GIT

```
<<<<<<< HEAD
    Entonces el código de respuesta es 200
=======
    Entonces el código de respuesta es 201
>>>>>>> develop
```

1. Abre el archivo en VS Code: aparecen **Accept Current**, **Accept Incoming** y **Accept Both**.
2. Deja el código como debe quedar y borra las marcas `<<<`, `===`, `>>>`.
3. Corre las pruebas.
4. `git add` del archivo, `git commit` y `git push`.

ERRORES TÍPICOS

`remote: Permission to …/canvia-restassured-bdd.git denied to otro-usuario. (403)`

CAUSA	SOLUCIÓN
Tu PC tiene guardada otra cuenta de GitHub (por ejemplo, la personal) y Git la usa sin preguntar.	`gh auth status` y `gh auth login` con la cuenta correcta. En Windows, además: `cmdkey /delete:git:https://github.com` y vuelve a hacer `push`.

`! [rejected] main -> main (fetch first) · o · protected branch hook declined`

CAUSA	SOLUCIÓN
Hay commits en GitHub que no tienes, o intentaste subir directo a una rama protegida.	Si es tu rama: `git pull` y luego `git push`. Si es `develop` o `main`: crea tu rama `feature/…` y abre un PR.

`error: Your local changes would be overwritten by checkout`

CAUSA	SOLUCIÓN
Quieres cambiar de rama con cambios sin guardar.	Haz commit, o guárdalos aparte: `git stash`, cambia de rama y luego `git stash pop`.

`Author identity unknown · Please tell me who you are`

CAUSA	SOLUCIÓN
Falta tu nombre y correo en Git.	Paso 3: `git config --global user.name` y `user.email`.

`Subí target/, node_modules/ o reports/ por error`

CAUSA	SOLUCIÓN
Usaste `git add .` con archivos generados.	`git rm -r --cached target` y commit. El `.gitignore` del repo ya los excluye.

NUNCA

`git push --force` en `develop` o `main`. Si crees que lo necesitas, pregunta primero al instructor.

PARA TENER A LA MANO

Chuleta de comandos

QUIERO…	COMANDO
Ver qué cambió	`git status` · `git diff`
Guardar cambios	`git add <archivos>` `git commit -m "feat: …"`
Subir mi rama	`git push`
Traer lo nuevo	`git pull`
Ver el historial	`git log --oneline --graph -15`
Ver mis ramas	`git branch`
Cambiar de rama	`git switch <rama>`

QUIERO…	COMANDO
Crear una rama	`git switch -c feature/…`
Descartar cambios de un archivo	`git restore <archivo>`
Sacar un archivo del commit	`git restore --staged <archivo>`
Corregir el último mensaje (sin push)	`git commit --amend`
Abrir un PR	`gh pr create --base develop`
Ver la CI de mi PR	`gh pr checks`
Saber con qué cuenta estoy	`gh auth status`

TAMBIÉN CON CLICS EN VS CODE

Vista **Source Control** (`Ctrl` + `Shift` + `G`): **+** junto a un archivo es `git add` ; mensaje y **Commit** es `git commit` ; **Sync Changes** es `pull` + `push` . La rama actual aparece abajo a la izquierda. Aprende primero los comandos: cuando algo falla, la terminal te dice exactamente qué pasó.

Checklist: ¿listo para entregar?

Configuración

- [ ] `git --version` responde.
- [ ] `git config --global --list` muestra tu nombre y correo.
- [ ] `gh auth status` dice tu usuario de GitHub.
- [ ] Clonaste el repositorio y lo abriste con `code .`

Primera entrega

- [ ] Creaste `feature/ejercicio-N-tu-nombre` desde `develop` .
- [ ] Las pruebas pasan en tu PC antes del commit.
- [ ] El PR va hacia `develop` y dice `Closes #N` .
- [ ] La CI está en verde y respondiste los comentarios.

VERSIÓN EN LÍNEA

Esta guía también está en cada repositorio del curso: `docs/04-git-y-github.md` , con enlaces y comandos para copiar.